

A fun and friendly programming challenge from Axiomatics

Welcome to Axiomatics!

We're really happy that you've decided to take on this programming challenge. Hopefully, it will allow you to get a sense of what Axiomatics is all about, and give us an opportunity to get to know a little bit about you.

We've put together a task that in various ways is representative of the kind of development work we're doing at Axiomatics, which will also serve to test your problem-solving skills.

The Scenario

At Axiomatics, we build complex systems where different teams own different parts of the UI. This task simulates a microfrontend based architecture where independent applications must coexist and share data in real-time.

Background

The assignment gives you the freedom to implement a solution according to your personal coding standards. Freedom is given to use any library of preference, but focus should be placed on using modern and often updated libraries. The solution needs to represent the best practices and the problem solving skills that you would leverage in a professional work environment.

What we want you to do

You are required to build a modular dashboard consisting of three distinct parts:

1. **The Orchestrator:** The "Shell" that handles global navigation, layout, and mounting logic.
2. **MFE-Form:** A micro frontend containing a data-entry form.
3. **MFE-Visualizer:** A micro frontend that displays the data graphically.

A user should be able to enter data in MFE-Form, navigate to MFE-Visualizer, and see that data reflected there. The state must persist or sync between these two separate applications.

Detailed Tasks

1. The Orchestrator (The Host)

- Handle the mounting of the two MFEs based on the current URL route.
- Implement the mechanism for sharing state between the two.

2. MFE-Form (Remote A)

- Build a form with at least 3-4 fields (e.g., Title, Value, Category, and Color).
- On submit/change, this data must be shared with the Orchestrator or a global store.

3. MFE-Visualizer (Remote B)

- A "Graphical Representation" of the data from MFE-Form.
- **Requirement:** This shouldn't just be text.

Technically, there are a number of things that could be done for the requested tasks, but we don't want an exhaustive or perfect solution, rather a sensible prioritization of the tasks and a plan to solve them. In order to keep the assignment time-boxed, we would like you to dedicate no more than a handful of hours to it. It is considered a must to:

- Provide sufficient documentation for running the project.
- Treat the project as if it was in a git repository

In case you prefer to not tackle some of the problems that may appear in the development process, please write your thoughts in the README for the incomplete tasks, commit all your changes and send the zipped project (including the .git folder) back to us.

Extra points:

- using the following stack:
 - React (Latest stable version)
 - TypeScript (Focus on shared types)
 - Tailwind CSS (For utility-first styling)
 - SingleSpa as MFE orchestration tool
 - Vite as build tool
- writing tests
- code comments